Seminar workshop

Codex for Research

Let the agent do the work. Keep the judgement.

Pann Phetra, College of Sustainability and Tourism
Ritsumeikan Asia Pacific University

Install

5 minutes, free account works

Two real projects

A thesis and a data case study

Your turn

A step-by-step guide on GitHub

A chatbot answers. An agent works in your folder.

Chatbot	Agent (Codex)
Replies in a chat window	Reads and writes files in one folder
You copy results out by hand	Saves results where you work
Cannot run anything	Runs code and checks the output
Forgets what it told you	Every change can be reviewed and undone

What you need

1

A ChatGPT account

Codex is in every plan, including Free. Limits vary.

2

The ChatGPT desktop app

macOS, Windows or Linux

3

One folder

Your project, or my example data

> Guide: github.com/Pann13223029/codex-seminar

Install in four steps

1	**Download** the ChatGPT desktop app	learn.chatgpt.com/docs/quickstart
2	**Sign in** with your ChatGPT account	Free works
3	**Open a folder**	Codex only sees this folder
4	**Choose Codex**, then New chat	In the ChatGPT dropdown

Live demo now. Optional: install Git so you can review and undo changes.

The way of working



Brief once

AGENTS.md: the project, the data, the rules

Plan before work

"Write a plan and wait for my OK"

Check every step

Small steps are easy to check and undo

AGENTS.md: the brief Codex reads first

```
# Project brief
- Question: does "4 in 10 plants generate"
  understate how much waste is covered?

## Files
- data/incinerators.csv: one row per plant-year

## Rules
- Plan first; wait for my OK.
- Never change the original data.
- Every number must come from code you ran.
- Never invent references or statistics.
```

Write it once

Codex reads it before every task in that folder

Put your rules in it

Your standards apply every time

Template in step 02

Copy, fill in, done

A prompt that works

```
I want to know [goal].
Use [which files]. Save results to [where].
Before changing anything, write a plan and tell me
how I can check each result. Wait for my OK.
```

Goal

The question, not the code

Files

Where to read and write

Check

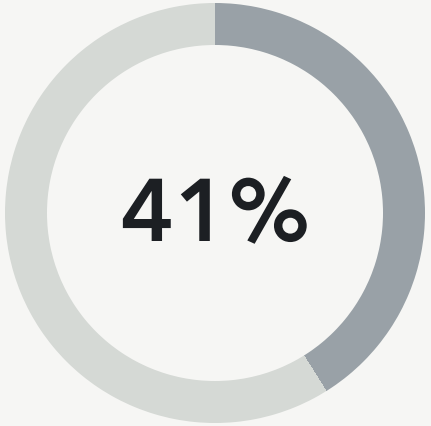
How you will verify

Stop

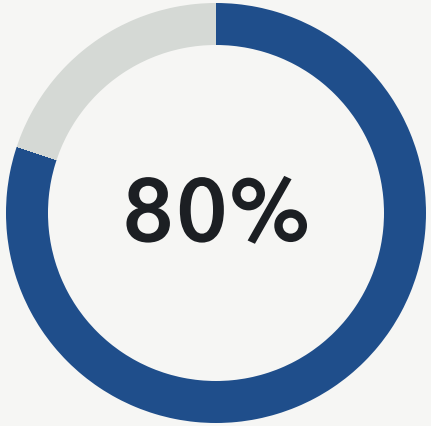
Wait for your OK

Example 1: my graduation thesis

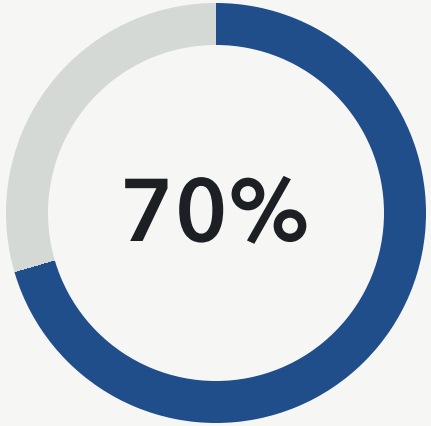
Japan says about 4 in 10 waste incinerators generate electricity. Is that low?



of plants



of waste



of capacity

20 years of Ministry spreadsheets, 23,593 records linked into 1,690 plant histories. FY2024 shown.

Who did what

The AI (Codex, Claude Code)	Me
Helped write the code that reads 20 spreadsheets	The question and what counts as an answer
Wrote and ran the statistics I specified	Every definition: what "generates" means
Built figures, tables and this kind of deck	Checking the numbers against the data
Drafted text from sources I had checked	What the results mean, and the final text

> Fast hands, not the brain. The thesis declares how AI was used.

Make mistakes hard to miss

Tests in the code

Plants that must link are checked every run; the result cannot change if the rows are shuffled

The build stops

If a number in the text no longer matches the results file, the thesis will not build

A mock examiner

An AI attacks the thesis, another defends it, I decide what to fix

> Ask the agent to check its own work, then check the check yourself.

Example 2: a data analytics capstone

Google Merchandise Store, one year of web analytics, 903,653 sessions in BigQuery. Which channels deserve the marketing budget?

41%

of revenue came from Google's own employees, hidden as "Referral"

98.8%

of the top 1% "likely buyers" in Google's own teaching model were Google staff

89%

of Display's credit came from one corporate buyer

Ask who is buying before you model

What worked

Asking "who is actually buying?" before modelling

Rebuilding Google's lab from its own code to test it

Checking my filter against the true source: 99.2% correct

What went wrong

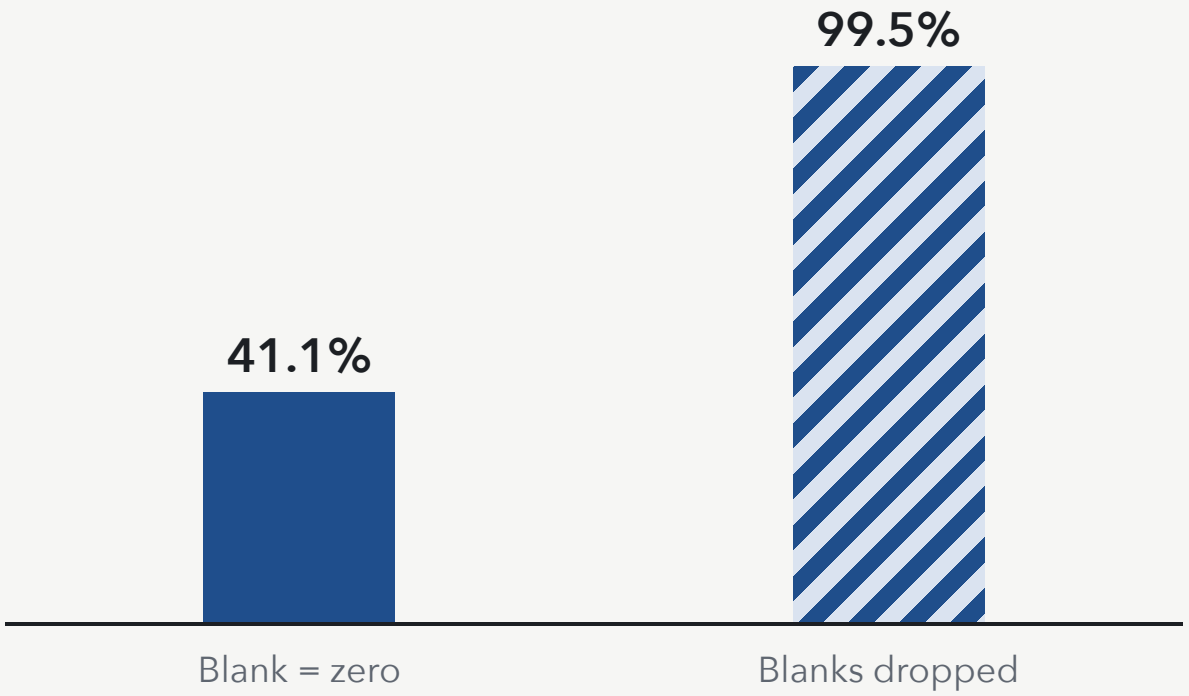
The AI stated facts about the lab from memory, twice wrongly

Caught only by checking the original source

Rule since then: verify before you present

> "Analysis designed and directed by me; code written with AI assistance."

One rule for blanks, two different answers



Share of FY2024 plants with a generator.

In 2024 the survey left 595 non-generators blank; in 2005 it wrote 0. Drop the blanks and you get 417 of 419.

Always ask: how were missing values treated?

Skills: save a workflow, reuse it

`.agents/skills/coverage-check/SKILL.md`

```
---
name: coverage-check
description: Calculate generator coverage
  for a fiscal year. Use when the user asks
  about coverage. Not for other analyses.
---
1. Read the definitions in AGENTS.md.
2. Calculate the three shares.
3. Tell the user one number to check by hand.
```

Make one

Type **$skill-creator** and describe the job

Use it

Type **$coverage-check**, or just ask; the description decides when it runs

Keep it small

One job per skill, checks built in

Your turn: the guide

00	Install and sign in
01	First task: create, review and undo a change
02	Your project: brief, plan, small steps, prompt templates
03	Verify and disclose
04	Make a skill
99	**No project?** Follow my example with copy-and-paste prompts

> github.com/Pann13223029/codex-seminar

Before you use any result

- ✓ Recompute two numbers by hand
- ✓ Ask how blanks were treated
- ✓ Check what each definition counts
- ✓ Check every reference exists
- ✓ Ask "what could be wrong?"
- ✓ Rewrite what you cannot explain

> Disclose it. APU treats undisclosed AI help as possible plagiarism. Template in step 03.

Four ways to get it wrong

Private data in the folder

Interviews, personal data, passwords: keep them out

Invented references

They look real. Search for every one

Trusting a confident answer

Ask for the source and the file behind each number

Handing over the thinking

The question and the meaning are yours

If you remember one thing

Let the agent do the work. Keep the judgement, and check before you trust.

The guide

github.com/Pann13223029/codex-seminar

Questions

Thank you